APOSTILA DO ALUNO — AULA 11

Conectividade e Interoperabilidade de Sistemas

Curso: Técnico em Desenvolvimento de Sistemas | Turma: 103513 | Disciplina: Modelagem de Sistemas (82h + 04h planejamento)

Data: 22/10/2026 (quinta-feira) | Horário: 07:30 às 11:05

Objetivos desta aula

- Diferenciar conectividade de interoperabilidade e reconhecer os 3 níveis de interoperabilidade (técnica, sintática e semântica).
- Explicar a arquitetura cliente-servidor e por que as regras de negócio ficam no servidor.
- Ler e escrever requisições de uma API REST: método HTTP, endpoint, corpo em JSON e código de status.
- Decidir quando uma integração é síncrona ou assíncrona e entender o webhook.
- Escrever requisitos não funcionais de conectividade e um plano B para cada sistema externo.
- Desenhar um diagrama de integração (componentes, sistemas externos e conversas) no draw.io.
- Encontrar problemas de segurança e de modelagem num diagrama de integração.

1. Conectividade e interoperabilidade

Quando você paga um lanche com PIX pelo app, pelo menos quatro sistemas conversam em poucos segundos: o app no celular, o servidor da lanchonete, o gateway de pagamento e o sistema do seu banco. Nenhum deles foi feito pela mesma equipe — e mesmo assim tudo funciona. Esta aula trata de como modelar essas conversas antes de programar. Até aqui olhamos para o sistema por dentro (Aulas 4 a 9) e para a pessoa que o usa (Aula 10); hoje olhamos para fora: com quem o sistema conversa.

1.1 Conectividade

Conectividade é a capacidade de um sistema se CONECTAR a outros e trocar dados por uma rede (internet, Wi-Fi, 4G). A pergunta da conectividade é: “os dois sistemas conseguem se alcançar?”. Sem conectividade, o app do TechBurger abre, mas não carrega o cardápio nem envia o pedido. Conectar é o primeiro passo — mas não garante que um sistema ENTENDA o outro.

1.2 Interoperabilidade

Interoperabilidade é a capacidade de sistemas DIFERENTES — muitas vezes de empresas diferentes — trabalharem juntos: eles trocam dados E entendem o que receberam, do jeito certo. O melhor exemplo do dia a dia é o PIX: qualquer app de banco paga para qualquer outro banco, porque todos seguem o mesmo padrão definido pelo Banco Central.

	Conectividade	Interoperabilidade
Pergunta	Os sistemas conseguem se ALCANÇAR?	Os sistemas conseguem se ENTENDER?
Depende de	Rede, internet, endereço do servidor	Formato e significado combinados
Analogia	A estrada e o telefone	A língua que os dois falam
Exemplo	O app chega ao servidor	O servidor entende o pedido que chegou

1.3 Os 3 níveis da interoperabilidade

Nível	O que precisa estar combinado	Se falhar…
Técnica	Os sistemas se conectam e trocam mensagens (internet + HTTP).	Nada chega: “sem conexão”.
Sintática	O mesmo FORMATO: JSON, com os mesmos nomes de campos.	Chega, mas não abre (“qtd” x “quantidade”).
Semântica	O mesmo SIGNIFICADO: reais ou centavos? “PAID” quer dizer “pago”?	Abre e é lido errado: 4180 centavos vira R$ 4.180,00.

O erro semântico é o mais perigoso: o sistema parece funcionar, não mostra erro nenhum — e cobra R$ 4.180,00 por um lanche.

2. Arquitetura cliente-servidor

2.1 Cliente e servidor

Quase todo sistema moderno tem duas pontas. O CLIENTE é o programa que a pessoa usa — o app no celular ou o site no navegador. O SERVIDOR é o computador que guarda os dados e executa as regras de negócio. O cliente faz uma REQUISIÇÃO (“quero o cardápio”) e o servidor devolve uma RESPOSTA. Um servidor atende milhares de clientes ao mesmo tempo. No TechBurger, o app é o cliente; os pedidos, os pontos de fidelidade e as regras ficam no servidor.

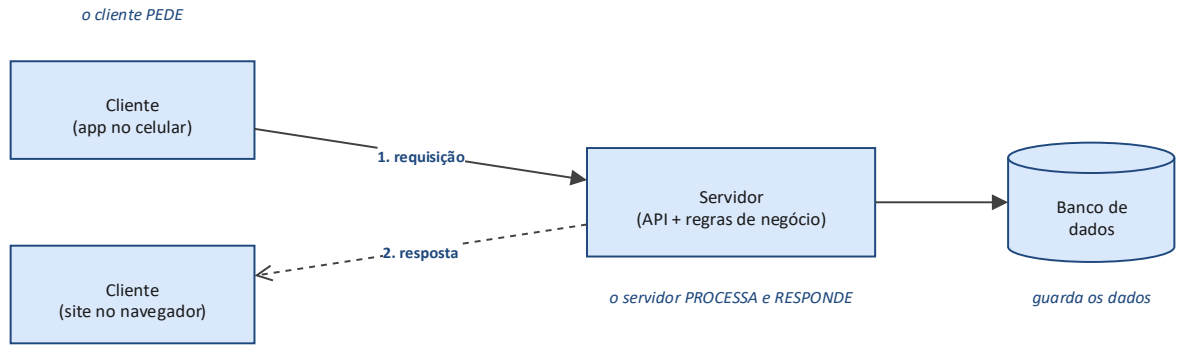


Figura 1 — Cliente-servidor: o cliente pede, o servidor processa, guarda e responde.

2.2 Uma analogia: o restaurante

No restaurante, você (o cliente) não entra na cozinha: faz o pedido ao garçom, num formato combinado — o cardápio. O garçom leva à cozinha (o servidor) e traz a resposta: o lanche, ou “acabou o pão”. Você não precisa saber COMO a cozinha funciona, só COMO pedir. O garçom faz o papel da API, que veremos na seção 3.

2.3 Por que as regras ficam no servidor

- **Segurança:** o app pode ser copiado e alterado por qualquer pessoa; o servidor fica sob o controle da empresa.
- **Uma regra só, para todos:** app, site e balcão usam o MESMO servidor. O pedido mínimo de R$ 15,00 (Aula 1) é conferido no servidor, mesmo que o app também avise.

- **Banco protegido:** o app nunca acessa o banco de dados direto; sempre pede ao servidor.
- **Segredos guardados:** chaves e senhas de outros sistemas (como a do gateway de pagamento) ficam SÓ no servidor.

3. HTTP, APIs e REST

3.1 O que é uma API

API (Application Programming Interface, interface de programação de aplicações) é a “porta de entrada” que um sistema oferece a outros. Ela é um CONTRATO: diz o que pode ser pedido, em que formato e o que volta. Quem usa a API não precisa conhecer o código de dentro do outro sistema. É assim que o TechBurger cobra PIX sem ser um banco: ele usa a API de um gateway de pagamento. Toda API séria tem uma DOCUMENTAÇÃO com a lista de pedidos possíveis e exemplos.

3.2 HTTP: requisição e resposta

HTTP é o conjunto de regras de requisição e resposta usado na web. HTTPS é o HTTP com criptografia: ninguém no caminho consegue ler os dados (cartão, CPF, endereço) — por isso toda conversa com dados pessoais usa HTTPS. Toda requisição tem um MÉTODO (o que fazer) e um ENDEREÇO (sobre o quê); pode levar CABEÇALHOS (quem está pedindo, em que formato) e um CORPO com os dados. Toda resposta tem um CÓDIGO DE STATUS (deu certo?) e, quase sempre, um corpo.

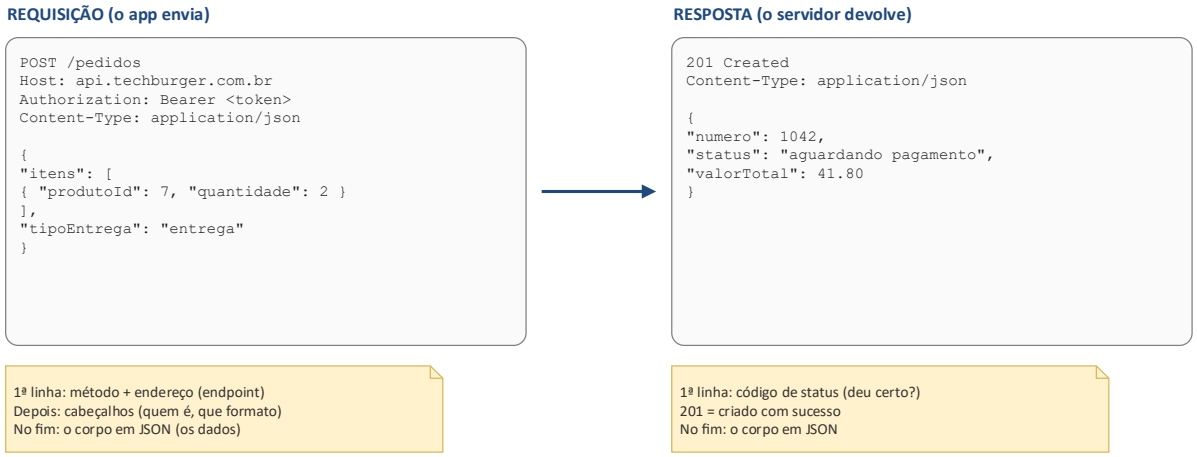


Figura 2 — Uma requisição real para criar um pedido no TechBurger e a resposta do servidor.

3.3 REST: tudo é um recurso

REST é o estilo de API mais usado hoje. Nele, cada “coisa” do sistema é um RECURSO com um endereço: /pedidos, /clientes, /produtos, /cobrancas (no plural). Um item específico ganha o número: /pedidos/1042. O endereço completo de um recurso é o ENDPOINT. Repare que os recursos são, quase sempre, as CLASSES do diagrama da Aula 5.

3.4 Os métodos HTTP

Método	Para quê	Exemplo no TechBurger
GET	Consultar, sem mudar nada	GET /cardapio · GET /pedidos/1042
POST	Criar algo novo	POST /pedidos (cria o pedido 1042)

PUT / PATCH	Alterar (PUT troca tudo; PATCH só uma parte)	PATCH /pedidos/1042 (muda só o status)
DELETE	Remover	DELETE /pedidos/1042/itens/2 (tira um item do carrinho)

Regra de ouro: GET nunca cria nem altera nada.

3.5 Alguns endpoints da API do TechBurger

Endpoint	O que faz	Regra ou aula ligada
GET /cardapio	Lista os produtos disponíveis	Item em falta → sugerir substituto (Aula 1)
POST /pedidos	Cria o pedido	Pedido mínimo de R$ 15,00 no app (Aula 1)
GET /pedidos/1042	Consulta o status do pedido	Visibilidade do status (Aula 10)
POST /pedidos/1042/combo-gratis	Resgata o combo grátis	Exemplo guiado da Aula 9
GET /clientes/88/pontos	Consulta os pontos de fidelidade	1 ponto a cada R$ 10,00 (Aula 1)

3.6 Códigos de status

Faixa	Significado	Exemplos
2xx	Deu certo	200 OK (consultou) · 201 Created (criou)
4xx	O problema está no PEDIDO	400 dados inválidos · 401 sem login · 404 não existe · 409 conflito (“Erro 409” da Aula 10: o pedido já tinha 1 combo grátis)
5xx	O problema está no SERVIDOR	500 erro interno · 503 fora do ar

O código de status é para o PROGRAMA. A pessoa nunca deve ver “Erro 409”: ela vê uma mensagem clara, que diz o que houve e o que fazer (heurística 9, Aula 10).

4. JSON e o contrato da API

4.1 JSON: o formato dos dados

JSON (JavaScript Object Notation) é um texto simples, legível por pessoas e por qualquer linguagem de programação. É o formato mais usado nas APIs REST — é ele que garante a interoperabilidade SINTÁTICA. As regras básicas:

- Pares “nome”: valor entre chaves { }; listas entre colchetes [ ].
- Texto entre aspas (“entrega”); número sem aspas (41.80); verdadeiro/falso: true / false.
- Números usam ponto e não levam “R$”: 41.80, e não “R$ 41,80”.
- Cada atributo da classe vira um campo do JSON — com o mesmo nome dos dois lados.

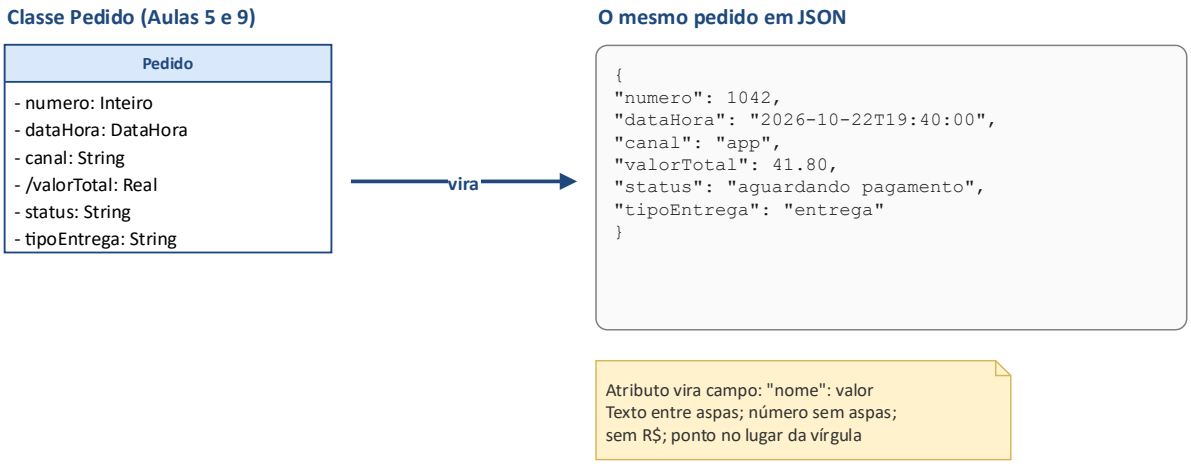


Figura 3 — A classe Pedido (Aulas 5 e 9) e o mesmo pedido em JSON.

4.2 O contrato da API

Antes de programar, os dois lados combinam o contrato de cada endpoint: o que ENVIA, o que RECEBE e os ERROS possíveis. O contrato é um requisito da integração. Se ele precisar mudar, cria-se uma nova versão (/v2/pedidos), para que quem usa a versão 1 não quebre.

Endpoint	Envia	Recebe (deu certo)	Erros possíveis
POST /pedidos	itens (produtoId, quantidade), tipoEntrega, endereço	201 + numero, status, valorTotal	400: abaixo de R$ 15,00 no app · 409: item em falta (com o substituto sugerido)

5. Integração síncrona x assíncrona

	Síncrona	Assíncrona
Como funciona	Quem chama ESPERA a resposta e só segue depois dela	Quem chama NÃO espera; o resultado chega depois
No diagrama	Ponta CHEIA (como na Aula 8)	Ponta ABERTA (como na Aula 8)
Exemplos	Pedir o QR Code do PIX; calcular a distância do endereço	Aviso de pagamento confirmado; push “saiu para entrega”; e-mail com recibo

Para decidir, faça a pergunta-chave: “preciso do resultado AGORA para continuar?”. Se sim, síncrona; se não, assíncrona. Pedir o QR Code é síncrono, porque o app só consegue mostrar o PIX com ele. Saber se o cliente pagou é assíncrono: pode levar segundos ou minutos. Enviar o push “pedido a caminho” também é assíncrono: o pedido não depende disso.

5.1 Webhook: “não me ligue, eu te ligo”

Em vez de perguntar ao gateway a cada segundo “já pagou?”, o servidor do TechBurger deixa um endereço de aviso. Quando o pagamento acontece, o gateway CHAMA esse endereço (por exemplo, POST /webhooks/pagamento): isso é um webhook. O webhook chega ao SERVIDOR — nunca direto no app do

cliente — e só então o pedido vai para a cozinha. É assim que a regra da Aula 1 ("só vai para a cozinha com pagamento confirmado") é garantida pela integração.

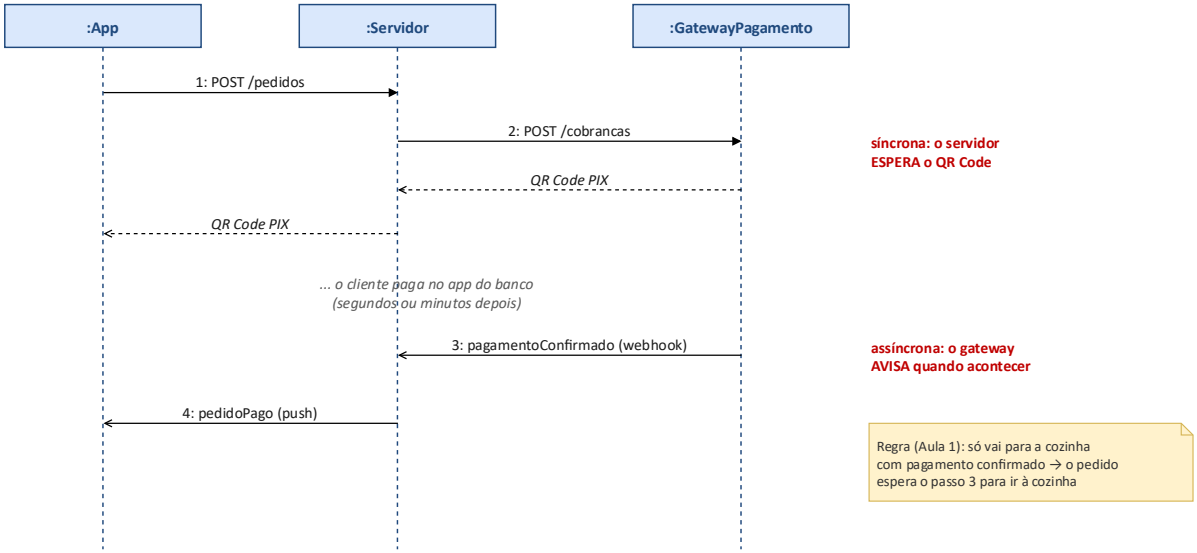


Figura 4 — O PIX visto como diagrama de sequência (Aula 8): chamadas síncronas e o webhook assíncrono.

6. Requisitos não funcionais de conectividade

Cada integração é um ponto que pode falhar. Por isso, cada seta para um sistema externo pede requisitos não funcionais (Aula 2) — e cada um deles pode virar critério de aceite e teste (Aula 3).

RNF	O que define	Exemplo no TechBurger
Disponibilidade	Quanto tempo fica no ar	"99,5% do mês" ≈ 3,6 horas fora do ar por mês
Tempo de resposta	Quanto esperar antes de considerar falha	"O gateway responde em até 3 segundos em 95% das vezes"
Segurança	Proteção dos dados e de quem acessa	HTTPS; login por chave ou token; chaves só no servidor; só os dados necessários (LGPD)
Escalabilidade	Aguentar os picos	"Sexta à noite, 3 vezes mais pedidos, sem lentidão"
Tolerância a falhas	O que fazer quando a outra ponta cai	Tentar de novo, avisar a pessoa, plano B

6.1 Plano B: e se a integração falhar?

Se falhar…	O que o TechBurger faz
Gateway de pagamento	O pedido NÃO vai para a cozinha; avisar a pessoa e oferecer pagar no balcão.
API de Mapas	Não liberar a entrega; oferecer retirada no balcão.
Serviço de notificações	O pedido segue normalmente; o aviso é tentado de novo depois.

A mensagem para a pessoa segue a UX da Aula 10: dizer o que houve e o que fazer — "Pagamento indisponível no app agora. Você pode pagar no balcão." Exemplo de critério de aceite: Dado que o gateway não respondeu

em 3 segundos, Quando o cliente finaliza o pedido, Então vê a mensagem para pagar no balcão. Outro: Dado um pedido não pago, Quando ele chega ao sistema da cozinha, Então NÃO aparece na tela.

6.2 Escolhendo a tecnologia pelo RNF

- **Pedido e consulta entre app e servidor:** API REST com HTTPS e JSON.
- **Aviso vindo de outro sistema (pagamento confirmado):** webhook.
- **Aviso no celular da pessoa:** notificação push, por um serviço externo.
- **Dispositivos (Internet das Coisas):** um sensor de temperatura da câmara fria envia leituras a cada minuto — protocolos leves como o MQTT foram feitos para isso.
- **O RNF decide:** precisa de resposta na hora? pode atrasar? quantas mensagens por minuto?

7. O diagrama de integração

7.1 Para que serve

O diagrama de integração mostra as PEÇAS do sistema e os sistemas EXTERNOS com que elas conversam. Cada seta é uma conversa: quem chama quem, o que pede e se espera a resposta. Ele é uma versão simplificada do diagrama de componentes da UML e ajuda a ver, de uma vez, onde estão os riscos — cada seta para fora pode falhar. Junto com ele vai a TABELA de integrações (tipo, RNF e plano B de cada seta).

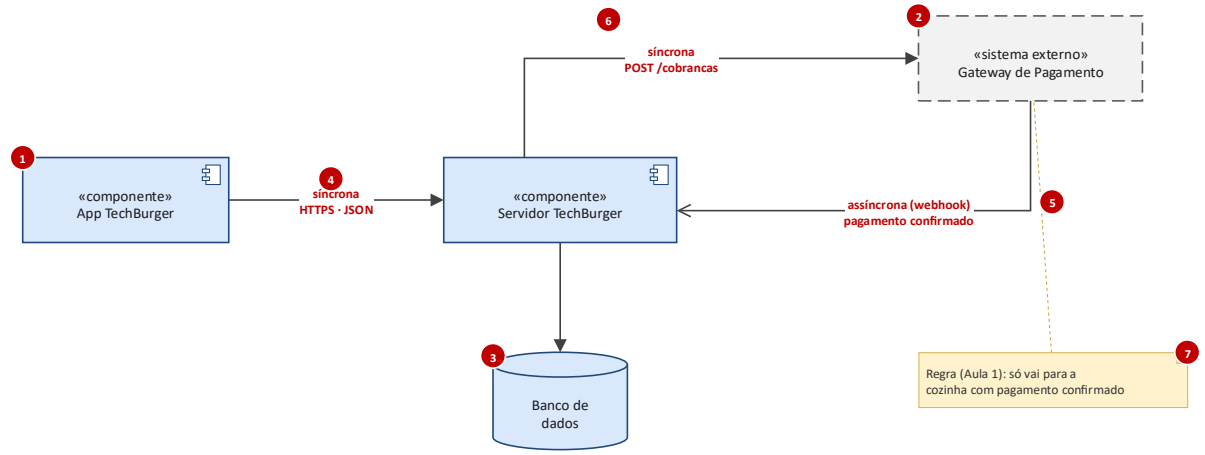


Figura 5 — Anatomia de um diagrama de integração: as 7 peças numeradas.

As peças se dividem em dois grupos. QUEM conversa: o componente (1), o sistema externo (2) e o banco de dados (3), além da nota (7) que explica regras, RNF e plano B. COMO conversam: a chamada síncrona (4), a mensagem assíncrona (5) e o rótulo da seta (6), que diz o tipo e o que é pedido.

7.2 As peças, uma a uma

Peça	Como desenhar	Exemplo
1. Componente	Retângulo com «componente» na 1ª linha (ícone de componente opcional). É do NOSSO sistema.	App TechBurger, Servidor TechBurger, Tela da Cozinha
2. Sistema externo	Retângulo de borda tracejada com «sistema externo». É de OUTRA empresa: pode mudar, atrasar ou cair.	Gateway de Pagamento, API de Mapas
3. Banco de dados	Cilindro. Só o servidor acessa.	Banco de dados do TechBurger

4. Chamada síncrona	Linha contínua, ponta CHEIA, saindo de quem chama.	App → Servidor: POST /pedidos
5. Mensagem assíncrona	Linha contínua, ponta ABERTA.	Gateway → Servidor: webhook "pagamento confirmado"
6. Rótulo da seta	Tipo + o que é pedido (endpoint) + protocolo, se quiser.	síncrona · GET /distancia · HTTPS · JSON
7. Nota	Nota amarela ligada à peça por linha tracejada.	"Regra (Aula 1): entrega só até 5 km. Plano B: retirada no balcão"

7.3 O que pode ser um sistema externo

Pagamento (gateway de PIX e cartão); localização (API de mapas: distância, rota, CEP); avisos (push, SMS, WhatsApp, e-mail); governo (emissão de nota fiscal eletrônica); parceiros (apps de delivery, outros sistemas da empresa). Na UML, eles aparecem como ATORES-SISTEMA no diagrama de casos de uso (Aula 4) — como o ator "Sistema de Pagamento" do TechBurger.

7.4 Como montar, em 5 passos

1. Comece pelos componentes do seu sistema: app, servidor, banco.
2. Releia as regras de negócio: cada regra que depende de fora aponta um sistema externo.
3. Desenhe as setas: a seta sai de quem CHAMA.
4. Para cada seta: síncrona ou assíncrona? Qual endpoint?
5. Acrescente as notas: a regra garantida, o RNF e o plano B de cada sistema externo.

7.5 Integração e a UML: o fio condutor

Na UML e nas aulas anteriores	No diagrama de integração (hoje)
Ator-sistema do caso de uso (Aula 4)	Sistema externo
Classe e atributos (Aula 5)	Recurso da API (/pedidos) e campos do JSON
Decisão do diagrama de atividades (Aula 7)	Código de status (400, 409) e plano B
Mensagem do diagrama de sequência (Aula 8)	Chamada da API, síncrona ou assíncrona
Tela e mensagem de erro (Aula 10)	Mensagem clara quando a integração falha
Requisito não funcional (Aula 2)	Disponibilidade, tempo de resposta e segurança de cada seta

8. Exemplo guiado: as integrações do TechBurger no draw.io

8.1 Onde fica cada coisa no draw.io

O diagrama de integração usa quase só a paleta Geral, que já vem aberta à esquerda. O componente UML com ícone é opcional: um retângulo com «componente» escrito na 1ª linha vale do mesmo jeito.

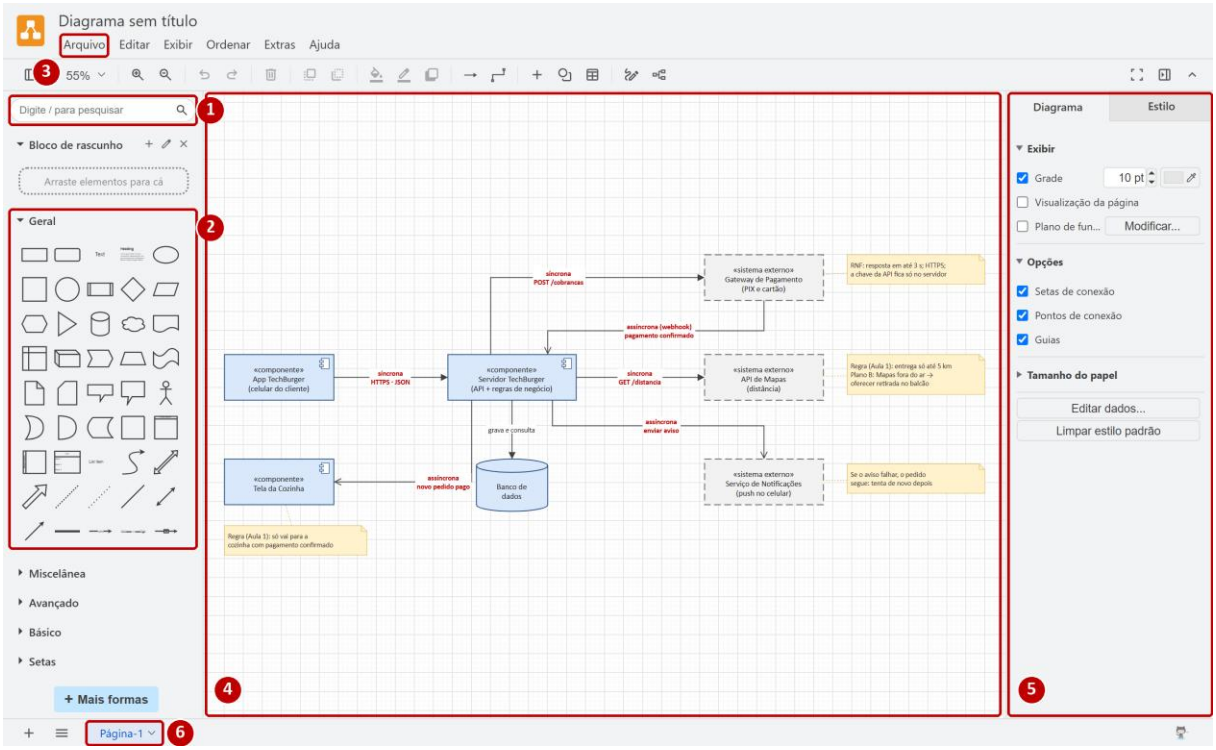


Figura 6 — A tela do draw.io: (1) pesquisa, (2) paleta Geral, (3) menu Arquivo, (4) área de desenho, (5) painel de formatação, (6) páginas.

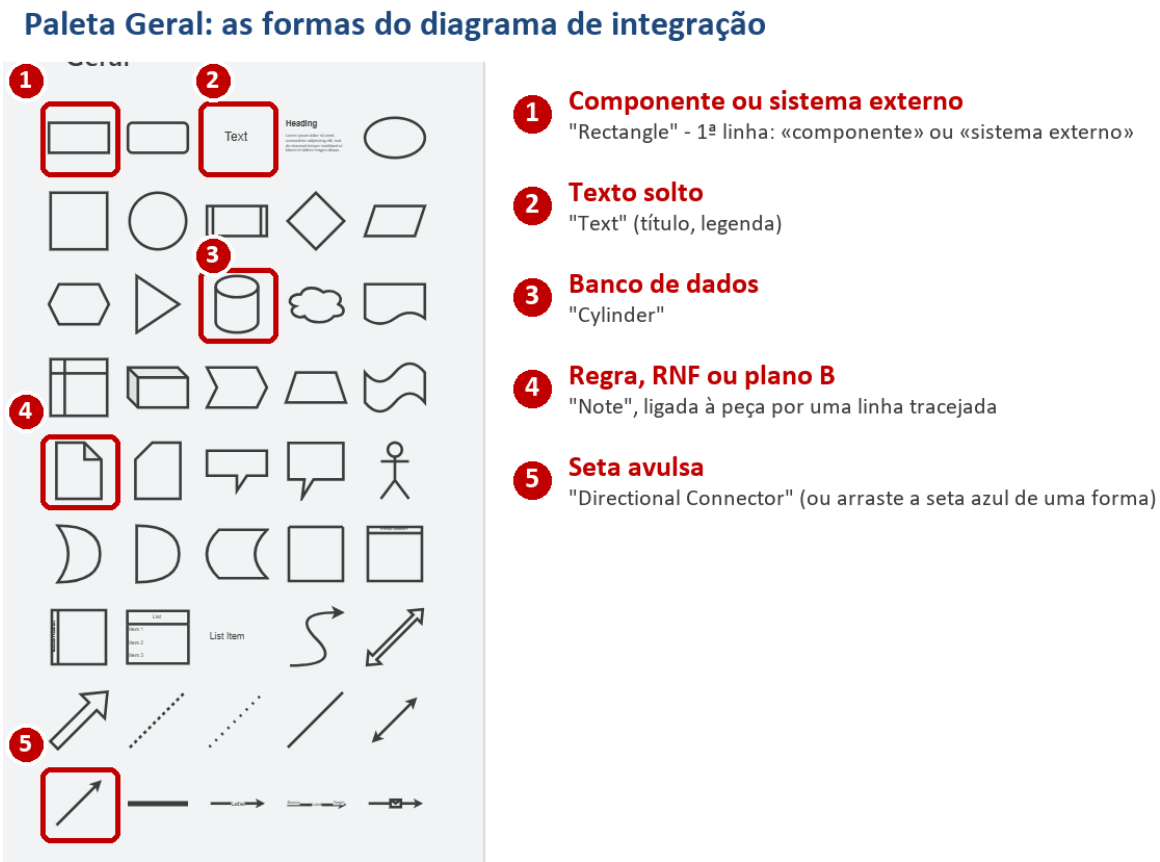


Figura 7 — As formas do diagrama de integração na paleta Geral.

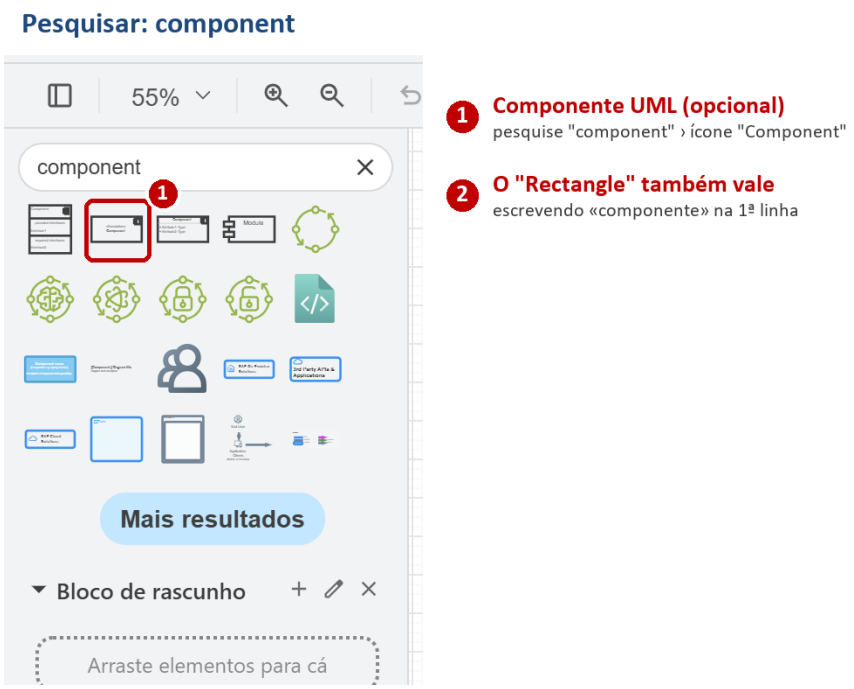


Figura 8 — Componente UML pela pesquisa (opcional).

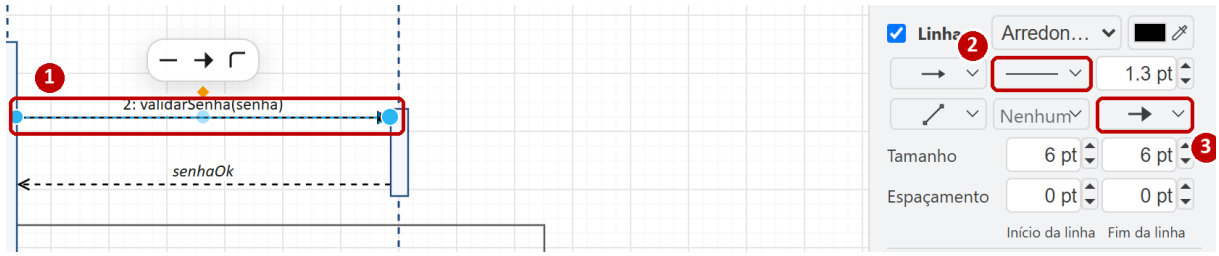


Figura 9 — Seta assíncrona: selecione a seta (1) e, no painel Estilo, troque o "Fim da linha" (3) para a ponta aberta; o padrão da linha (2) deixa tracejada a linha da nota.

Peça	Onde fica no draw.io
Componente	Geral › "Rectangle"; duplo clique e escreva «componente» + o nome (ou pesquise "component").
Sistema externo	Outro "Rectangle" (Ctrl+D duplica); Estilo › Linha › tracejada; escreva «sistema externo» + o nome.
Banco de dados	Geral › "Cylinder".
Seta	Passe o mouse na forma e arraste a seta azul até a outra forma (ou Geral › "Directional Connector").
Rótulo da seta	Duplo clique na seta; Shift+Enter quebra a linha.
Seta assíncrona	Selecione a seta › Estilo › Fim da linha › ponta aberta.
Nota	Geral › "Note", ligada à peça por uma linha tracejada.

- A seta precisa "grudar" na forma (contorno azul), senão não acompanha quando você move a forma.
- Para mover só o rótulo da seta, arraste o losango amarelo do texto.
- Ctrl+Z desfaz; Delete apaga.
- Salvem como "Integracao_TechBurger.drawio"; para exportar a imagem: Arquivo › Exportar como › PNG.

8.2 Passo a passo

Caso: o cliente faz um pedido pelo app do TechBurger e paga com PIX. Os passos estão nos slides 57 a 64: em cada um, o que entra no diagrama aparece em laranja.

Passo 0 — Ponto de partida: Arquivo › Abrir › “Aula 11 - Exemplo Guiado (ponto de partida).drawio”. Já estão lá o App TechBurger, o Servidor TechBurger e a seta síncrona entre eles (o app espera o número do pedido).

Passo 1 — Banco de dados: “Cylinder” embaixo do Servidor, com a seta “grava e consulta” saindo do Servidor. A seta não sai do App: só o servidor acessa o banco.

Passo 2 — Gateway de pagamento: sistema externo tracejado à direita; seta síncrona do Servidor, POST /cobrancas. É síncrona porque o servidor precisa do QR Code para mostrar ao cliente.

Passo 3 — O aviso do pagamento: seta de ponta aberta do Gateway para o Servidor: assíncrona (webhook), pagamento confirmado. O cliente pode levar minutos para pagar no app do banco, e o aviso vai ao Servidor porque é lá que estão a regra e a cozinha.

Passo 4 — API de Mapas: outro sistema externo; seta síncrona do Servidor, GET /distancia — a regra da entrega até 5 km depende dela.

Passo 5 — Tela da Cozinha: «componente» (é nosso); seta assíncrona do Servidor, novo pedido pago. O pedido só aparece na cozinha depois do aviso do passo 3.

Passo 6 — Serviço de Notificações: sistema externo; seta assíncrona do Servidor, enviar aviso. Se o push falhar, o pedido não para.

Passo 7 — As notas: Cozinha: “Regra (Aula 1): só vai para a cozinha com pagamento confirmado”. Gateway: RNF — resposta em até 3 s, HTTPS, chave só no servidor. Mapas: entrega até 5 km e plano B (retirada no balcão). Notificações: se falhar, o pedido segue.

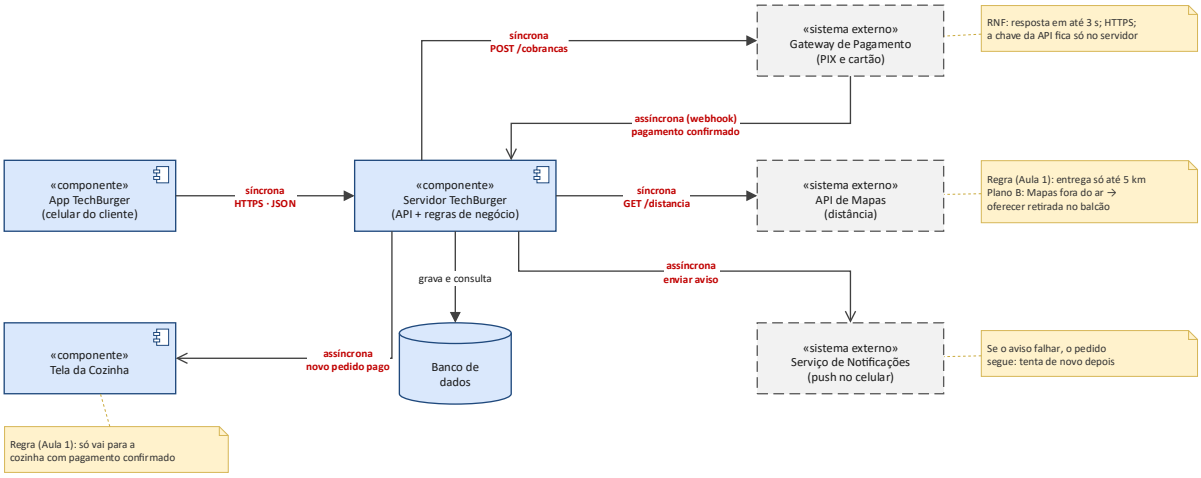


Figura 10 — Resultado do exemplo guiado: as integrações do TechBurger.

Checklist do exemplo guiado:

- Componentes (nossos) e sistemas externos (tracejados) estão separados?
- Só o Servidor acessa o banco e os sistemas externos?

- Toda seta tem tipo (síncrona / assíncrona) e o que pede?
- O pagamento confirmado chega por webhook ao Servidor?
- Cada sistema externo tem nota de regra, RNF ou plano B?

8.3 Revisão: do pedido ao aviso no celular

Depois do exemplo guiado, vale rever cada conceito da aula com o TechBurger (slides 67 a 73).

Cliente-servidor — o App é o cliente e pede; o Servidor processa as regras e responde. Só o Servidor fala com o banco e com os sistemas externos, e as chaves das APIs ficam nele (Figura 1).

HTTP e API — requisição = método + endereço (+ corpo); resposta = código de status (+ corpo). GET consulta, POST cria, PATCH altera, DELETE remove. No guiado: POST /pedidos, POST /cobrancas, GET /distancia (Figura 2).

JSON — o atributo da classe vira campo; mesmo formato e nomes = interoperabilidade sintática; mesmo significado = semântica (Figura 3).

Síncrona x assíncrona — síncrona quando preciso da resposta agora (QR Code, distância); assíncrona quando o resultado vem depois (pagamento confirmado, push). O webhook avisa o Servidor quando o PIX é pago (Figura 4).

RNF e plano B — cada seta para fora é um risco; o Gateway tem RNF de 3 s e HTTPS; Mapas fora do ar leva à retirada no balcão; a regra da cozinha é garantida pelo webhook (Figura 10).

Como tudo se liga — ator-sistema (Aula 4) → sistema externo; atributos (Aula 5) → JSON; decisões (Aula 7) → códigos de status; mensagens (Aula 8) → chamadas da API; a falha vira mensagem clara na tela (Aula 10); RNF (Aula 2) em cada seta.

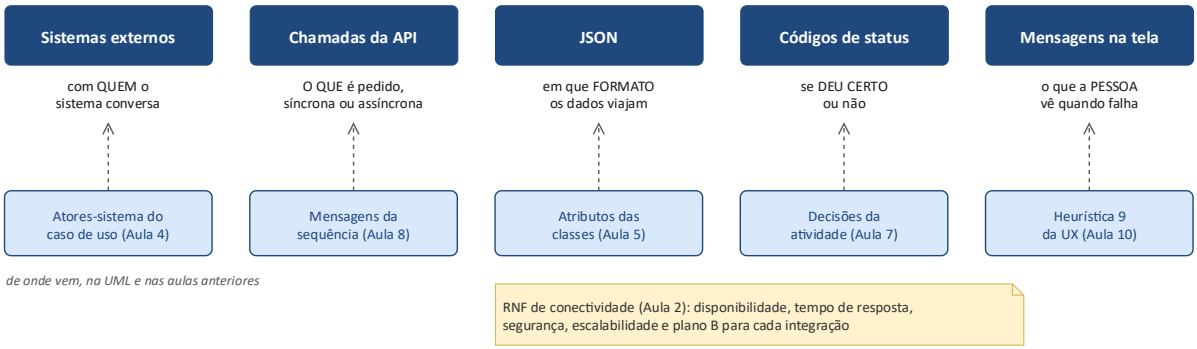


Figura 11 — Revisão: de onde vem cada parte da integração.

9. Exercício: encontre os problemas

Em duplas, analisem o diagrama de integração que um estagiário fez para o TechBurger. Ele tem 6 problemas: de segurança, de tipo de conversa (síncrona/assíncrona), de método HTTP e de quem fala com quem. Marquem cada um, digam POR QUE está errado e escrevam a correção. Tempo: 6 minutos, sem consultar a Apostila nos primeiros 5.

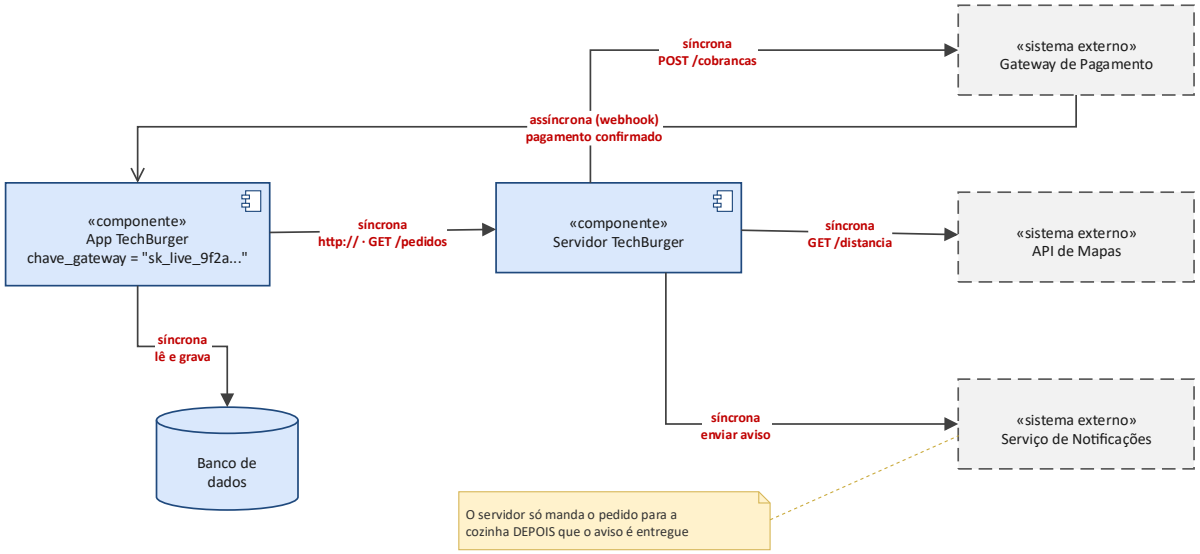


Figura 12 — O diagrama do estagiário: 6 problemas de integração.

Dica: percorram as setas uma a uma e perguntem: quem chama quem? Precisa esperar? O método combina com a ação? Algum segredo ou dado está exposto? Depois confiram com a seção 2.3 e a tabela da seção 3.4.

10. Erros comuns em integração

- Deixar segredos (chaves, senhas) no app do cliente.
- O app falar direto com o banco ou com sistemas externos.
- Tudo síncrono: um aviso lento trava o pedido inteiro.
- Seta sem rótulo: ninguém sabe o que vai nem quem espera.
- Esquecer o plano B: “e se o gateway cair?” sem resposta.
- Não combinar o SIGNIFICADO dos dados (centavos x reais, “PAID” x “pago”).

11. Boas práticas

- Comecem pelas regras de negócio: cada regra que depende de fora aponta uma integração.
- Toda seta com tipo, endpoint e protocolo (HTTPS · JSON).
- Síncrona só quando precisa da resposta AGORA; o resto, assíncrona.
- Para cada sistema externo: um RNF e um plano B.
- Documentem o contrato (o que vai, o que volta, os erros) antes de programar.

Glossário

Termo	Significado
Conectividade	Capacidade de um sistema se conectar a outros e trocar dados por uma rede.
Interoperabilidade	Capacidade de sistemas diferentes trocarem dados e os entenderem corretamente (níveis técnico, sintático e semântico).
Cliente / servidor	Programa que a pessoa usa e que faz os pedidos / computador que guarda os dados, executa as regras e responde.
API	Contrato de acesso a um sistema: o que pode ser pedido, em que formato e o

	que volta.
Endpoint	Endereço de um recurso da API (ex.: /pedidos/1042).
HTTP / HTTPS	Protocolo de requisição e resposta da web / o mesmo, com criptografia.
REST	Estilo de API em que cada coisa é um recurso com endereço, manipulado por GET, POST, PUT/PATCH e DELETE.
JSON	Formato de texto com pares “nome”: valor, usado para os dados nas APIs.
Código de status	Número da resposta HTTP que diz se deu certo (2xx), se o erro foi no pedido (4xx) ou no servidor (5xx).
Síncrona / assíncrona	Quem chama espera a resposta / não espera; o resultado chega depois.
Webhook	Aviso que um sistema externo envia ao nosso servidor quando algo acontece.
Sistema externo	Sistema de outra empresa, usado pela API dele.

Perguntas de revisão

1. Qual a diferença entre conectividade e interoperabilidade?
2. Quais são os 3 níveis de interoperabilidade? Dê um exemplo de falha em cada.
3. Por que as regras de negócio e as chaves ficam no servidor e não no app?
4. Qual método HTTP usar para criar, consultar, alterar e remover? Dê exemplos do TechBurger.
5. Como decidir se uma integração é síncrona ou assíncrona?
6. O que é um webhook? Onde ele aparece no pagamento PIX?
7. Cite 3 RNF de conectividade e um plano B para o gateway de pagamento.

Quiz relâmpago para casa

Respostas no Gabarito.

1. O que significa o código 201? E o 503?
2. Qual método HTTP nunca deve alterar dados?
3. “total_centavos”: 4180 lido como R$ 4.180,00 é falha de qual nível?
4. Ponta cheia ou aberta para o aviso “pedido saiu para entrega”?
5. No diagrama de integração, como se desenha um sistema de outra empresa?